

CMP4272 DATA STRUCTURES AND ALGORITHMS

Technical Report



STUDENT NAME: Amogh Dath Kalasapura Arunkumar

STUDENT ID: 24168333

DATE: May 16, 2025

Word Count: 3831 (Including code snippet)

Module Title:	Data Structures and Algorithms
----------------------	--------------------------------

Module Code:	CMP4272	
Assessment Title:	Technical Report	
Assessment Type:	CWRK	Weighting: 50%
College:	College of Computing	
Release date:	25/03/2025	
Submission date:	16/05/2025	

PYTHON ENVIRONMENT: USE SPYDER

Number	Component	Description	Materials	Marks
A1	Data Structures and Algorithmic Solution Algorithm Correctness	Effective use of data structures which suit the problem and appropriate. Efficient algorithm implementation with consideration of algorithm ethics. Testing	Refer to Lecture/Labs Week-7, Week-8 and Week-9.	12
A2	Algorithm Design - Process, Plans and Strategy	Discuss about the algorithm process, strategy, plans with regards to each of the algorithms you have implemented.	Refer to Lecture/Labs Week-7, Week-11	10
A3	Data Structures and Algorithmic Solution	Data Representation	Refer to Lecture/Labs Week-7, Week-11	3
B1	Algorithm Efficiency- Running time computation using Dynamic and Static Analysis	Computer running time for algorithms Compare and discuss results of dynamic analysis. Use Big-Oh, Big-theta and Big-omega. Sorting algorithms	Refer to Lecture/Labs Week-9 and Week-10	18
B2	Data Structures and Algorithmic Solution Algorithm Efficiency- Static Analysis	Graphs Big-Oh Searching algorithms	Refer to Lecture/Labs Week-8 and Week-10	4
B3	Data Structures and Algorithmic Solution	Searching, Strategies, Approaches	Refer to Lecture/Labs Week-8 and Week-10	3
		TOTAL		50

SECTION A: ALGORITHM DESIGN, IMPLEMENTATION AND CORRECTNESS [25 marks]

A1. Algorithm Implementation and Testing [12 marks]

The given student management system allows various operations like inserting, searching, sorting, display etc. You are required to add more functionality. All these three methods should be thoroughly tested (use file **Testing.py**) and then incorporated within the student management system (use files **StudentManagement.py** and **Main.py**)

A1.1 Write a method `search_by_name` which should return all students with the given name.

```
def search_by_name(self, name):  
    """  
    Function To search for students by name in the tree.  
  
    Parameters:  
    name (str): The name of the student to search for.  
  
    returns:  
    list: A list of student names that match the search  
criteria.  
    """  
    all_names = []  
    def _search(node):  
        if node is None:  
            return  
        # Check if the name matches the current node's name  
        if node.name.lower() == name.lower():  
            all_names.append(node)  
  
        _search(node.left)  
        _search(node.right)  
  
    _search(self.root)  
    return all_names
```

CodeBlock.1. Represents the code to search for students by name.

```
def test_search_by_name():  
  
    # Test case 1: Search for a name with a single match  
    name1 = "Ethan Hunt"  
    result = bst.search_by_name(name1)  
    result_formatted = [(student.student_id, student.name,  
student.gpa) for student in result]  
    print(f"Matching students for '{name1}': {result_formatted}") #  
# Expected Output: [(101, "Ethan Hunt", 3.2)]
```

Test Case.1. For A1.1

```
# Test case 2: Search for a name with multiple matches
name2 = "Alice Johnson"
result = bst.search_by_name(name2)
result_formatted = [(student.student_id, student.name,
student.gpa) for student in result]
# Expected: e.g., [(103, "Alice Johnson", 3.3), (105, "Alice
Johnson", 3.5)]
print(f"Matching students for '{name2}': {result_formatted}")
```

Test Case.2. For A1.1

```
# Test case 3: Search for a name with no match
name3 = "Zoe Smith"
result = bst.search_by_name(name3)
result_formatted = [(student.student_id, student.name,
student.gpa) for student in result]
print(f"Matching students for '{name3}': {result_formatted}")

# Expected: []
```

Test Case.3. For A1.1

A1.2 Write a method `total_student_count` that returns the total number of students.

```
def total_student_count(self):
    """Function to count the total number of students in the
    tree.

    parameters: None

    returns:
    int: The total number of students in the tree.
    """
    def _count(node):
        if node is None:
            return 0
        return 1 + _count(node.left) + _count(node.right)

    return _count(self.root)
```

CodeBlock.2. Represents the code to count the total number of students in the tree.

```
bst = StudentBST()
insert_sample_students(bst) # This should insert students into the
BST

def test_total_student_count():
    # Test case 1: Count students in the full tree
    result = bst.total_student_count()
    print(f"Total student count: {result}") # Expected Output: 12
```

Test Case.1. For A1.2

```
# Test case 2: Inserting a new student and counting again
bst.insert(116, "Mary Jane", 3.8)
result = bst.total_student_count()
print(f"Total student count after adding a new student:
{result}")
```

```
# Expected Output: 13
```

Test Case.2. For A1.2

```
# Test case 3: Count students in an empty tree
empty_bst = StudentBST() # Create a new empty BST
result = empty_bst.total_student_count()
print(f"Total student count in an empty tree: {result}")
```

```
# Expected Output: 0
```

Test Case.3. For A1.2

A1.3 Write an efficient method `display_ordered` displays the data ordered by ID.

```
def display_ordered(self):
    """Function to Sort student records using Merge Sort and
    displays them.

    Parameters:- none

    Returns :- list of students with their student id in order

    """
    students = []
    self._collect_students(self.root, students)

    sorted_students = self._merge_sort(students)

    for student in sorted_students:
        print(f"ID: {student.student_id}, Name: {student.name},
        GPA: {student.gpa}")
```

CodeBlock.3. Represents the code to sort student record using merge sort and as a result displays them.

```
def test_display_ordered():
    # Test case 1: Display sorted students in the full tree
    print("Test case 1: Display sorted students in the full tree")
    bst.display_ordered()
```

```
# Expected Output:
```

```
# ID: 101, Name: Ethan Hunt, GPA: 3.2
# ID: 102, Name: Bob Smith, GPA: 3.8
# ID: 103, Name: Kyle Johnson, GPA: 3.3
# ID: 104, Name: Hannah Lee, GPA: 3.6
# ID: 105, Name: Alice Johnson, GPA: 3.5
# ID: 106, Name: Liam Scott, GPA: 3.75
# ID: 107, Name: Diana Prince, GPA: 3.7
# ID: 108, Name: George Miller, GPA: 3.1
# ID: 109, Name: Jessica Davis, GPA: 3.0
# ID: 110, Name: Charlie Brown, GPA: 2.9
```

```
# ID: 112, Name: Ian Wright, GPA: 3.4
# ID: 115, Name: Fiona Clark, GPA: 3.9
# ID: 116, Name: Mary Jane, GPA: 3.8
# ID: 117, Name: Amogh Dath, GPA: 5.0
```

Test Case.1. For A1.3

```
# Test case 2: Insert a new student and display sorted students
print("\nTest case 2: Insert a new student and display sorted
students")
bst.insert(100, "Mary Jane", 3.8)
bst.display_ordered()

# Expected Output:
# ID: 100, Name: Mary Jane, GPA: 3.8
# ID: 101, Name: Ethan Hunt, GPA: 3.2
# ID: 102, Name: Bob Smith, GPA: 3.8
# ID: 103, Name: Kyle Johnson, GPA: 3.3
# ID: 104, Name: Hannah Lee, GPA: 3.6
# ID: 105, Name: Alice Johnson, GPA: 3.5
# ID: 106, Name: Liam Scott, GPA: 3.75
# ID: 107, Name: Diana Prince, GPA: 3.7
# ID: 108, Name: George Miller, GPA: 3.1
# ID: 109, Name: Jessica Davis, GPA: 3.0
# ID: 110, Name: Charlie Brown, GPA: 2.9
# ID: 112, Name: Ian Wright, GPA: 3.4
# ID: 115, Name: Fiona Clark, GPA: 3.9
# ID: 116, Name: Mary Jane, GPA: 3.8
# ID: 117, Name: Amogh Dath, GPA: 5.0

if __name__ == "__main__":
    test_display_ordered()
```

Test Case.2. For A1.2

A2. Algorithm Design [10 marks]

Discuss about the algorithm process with regards to the following:

A2.1 Tree - Insert node (insert) [Refer to StudentManagement.py]

```
def insert(self, student_id, name, gpa):  
    """Insert a student record into the tree """  
    new_node = StudentNode(student_id, name, gpa)  
    if self.root is None:  
        self.root = new_node  
        return  
  
    current = self.root  
    while True:  
        if student_id < current.student_id:  
            if current.left is None:  
                current.left = new_node  
                return  
            current = current.left  
        else:  
            if current.right is None:  
                current.right = new_node  
                return  
            current = current.right
```

The provided code implements a binary search tree (BST) insertion algorithm for managing student records (**student_id, name, gpa**). The algorithm follows a structured process to insert a new node into the tree while maintaining the BST property (all nodes to the left of a node have a smaller key, and all nodes to the right have a larger key). Below are the key steps, each explained with accuracy and depth.

1.Initialization and Base Case Handling

Step:

The method `insert(self, student_id, name, gpa)` begins by creating a new node (`new_node`) **with** the provided `student_id`, `name`, and `gpa` using the `StudentNode(student_id, name, gpa)` **constructor**.

Explanation:

If the tree **is empty** (i.e., `self.root is None`), the **new** node becomes the root. This **is** the base **case**, ensuring the algorithm can handle an **empty** tree. The **method returns the new node to confirm insertion**.

Insight:

This step establishes **the foundation for the tree**, ensuring a starting point **for** subsequent insertions.

2.Traversal to the Insertion Point

Step:

A **while True** loop is used **with** `current = self.root` to traverse the tree. The **loop** continues **until** the **new** node **is** inserted.

Explanation:

The algorithm compares **the** `student_id` **of the** **new** node **with the** `current.student_id`. If `student_id < current.student_id`, **it** moves **to** **the** left subtree; otherwise, **it** moves **to the** right subtree.

Insight:

This leverages **the** BST property, ensuring efficient navigation ($O(h)$ **time** complexity, where h is **the** height **of the** tree) **to** find **the** correct insertion position.

3.Insertion into the Left Subtree

Step:

If `current.left` **is** **None** (indicating **no** left child), the `new_node` **is** assigned **to** `current.left`, **and** the **loop** breaks implicitly **by** **returning**.

Explanation:

This occurs **when** the traversal reaches a leaf node **where** the left child **is** vacant, **and** the **new** node's `student_id` **is less than the** `current node's student_id`.

Insight:

The use **of** `return current.left` **after** assignment ensures **the** algorithm terminates **and** provides **a** reference **to** **the** newly inserted node, maintaining tree integrity.

4.Insertion into the Right Subtree

Step:

If `current.right` **is** **None** (indicating **no** right child), the `new_node` **is** assigned **to** `current.right`, **and** the **loop** breaks implicitly **by** **returning**.

Explanation:

This happens **when** the **new node's student_id** is **greater than or equal to the current node's student_id**, **and** the right child position **is** available.

Insight:

The symmetry **between** left **and** right insertion logic ensures balanced handling **of** the BST property, though the code assumes **unique student_id values** (no duplicates).

5.Update of Current Node for Continued Traversal

Step:

After deciding the direction (left **or** right), **current** **is** updated to **current.left** **or** **current.right** **to continue** the traversal.

Explanation:

This step moves **the** algorithm deeper **into the** tree, recursively exploring **until an empty** slot is found.

Insight:

The infinite **True While** loop relies on the return statements to **exit**, which is a deliberate design choice to avoid explicit loop termination conditions.

The algorithm inserts a student record into a binary search tree iteratively. It creates a new node with `student_id`, `name`, and `gpa`, sets it as the root if the tree is empty, or traverses the tree using a `while True` loop to find the correct spot. The node is inserted as a left or right child based on `student_id`, maintaining the BST property with $O(h)$ time complexity, assuming unique keys and avoiding recursion.

A.2.2 Graph – Create Edge (`add_friendship`) [Refer to `StudentFriendshipGraph.py`]

```
def add_friendship(self, student1, student2):
    """Create an undirected edge (friendship) between two students."""
    if student1 not in self.graph:
        self.add_student(student1)
    if student2 not in self.graph:
        self.add_student(student2)

    self.graph[student1].append(student2)
    self.graph[student2].append(student1)
```

The provided code implements the `add_friendship(self, student1, student2)` method to create an undirected edge (friendship) between two students in a graph using an

adjacency list representation. The graph models social relationships where each student is a node and friendships are edges. Here are the key steps, explained simply:

1. Check and Add student1 to Graph

Step:

```
Verify if student1 exists in self.graph; if not, add it using self.add_student(student1).
```

Explanation:

This ensures student1 is a node with an empty friend list if it's new.

Insight:

This step prepares the graph for the edge by guaranteeing the node exists

2. Check and Add student2 to Graph

Step:

```
Verify if student2 exists in self.graph; if not, add it using self.add_student(student2).
```

Explanation:

This ensures student2 is a node with an empty friend list if it's new.

Insight:

Symmetry with step 1 ensures both nodes are ready for the undirected edge.

3. Create Bi-directional Edge (Friendship)

Step:

```
Append student2 to self.graph[student1] and student1 to self.graph[student2].
```

Explanation:

```
This adds student2 to student1's friend list and vice versa,
establishing a mutual (undirected) friendship.
```

Insight:

```
The dual appends reflect the undirected nature, ensuring both
students recognize the relationship.
```

The approach uses an adjacency list (`self.graph = {student_id: [list_of_friend_ids]}`) to represent the graph, which is highly relevant for modeling sparse social networks where friendships are fewer than possible connections. This structure supports efficient edge additions and is ideal for analyzing student relationships.

The `add_friendship(self, student1, student2)` method uses an adjacency list to model student friendships as undirected edges. It checks and adds `student1` and `student2` as nodes if absent, then appends each to the other's friend list, ensuring mutual connections. With $O(1)$ complexity for edge addition and $O(n + e)$ space usage, it's efficient for sparse social networks and supports analyzing relationships.

A3. Data Organization Diagram [3 marks]

Draw the sequence of tree diagrams showing the organization of the 12 student objects (refer to `insert_sample_students` function in `Main.py`) added to the tree structure with student ID being displayed in each node.

Detailed Steps:

The `insert` method of the `StudentManagement` class inserts each student object in the tree.

```
def insert(self, student_id, name, gpa):
```

Track the function and show insertion of each node in tree diagram.

The final diagram will be the tree having all 12 nodes

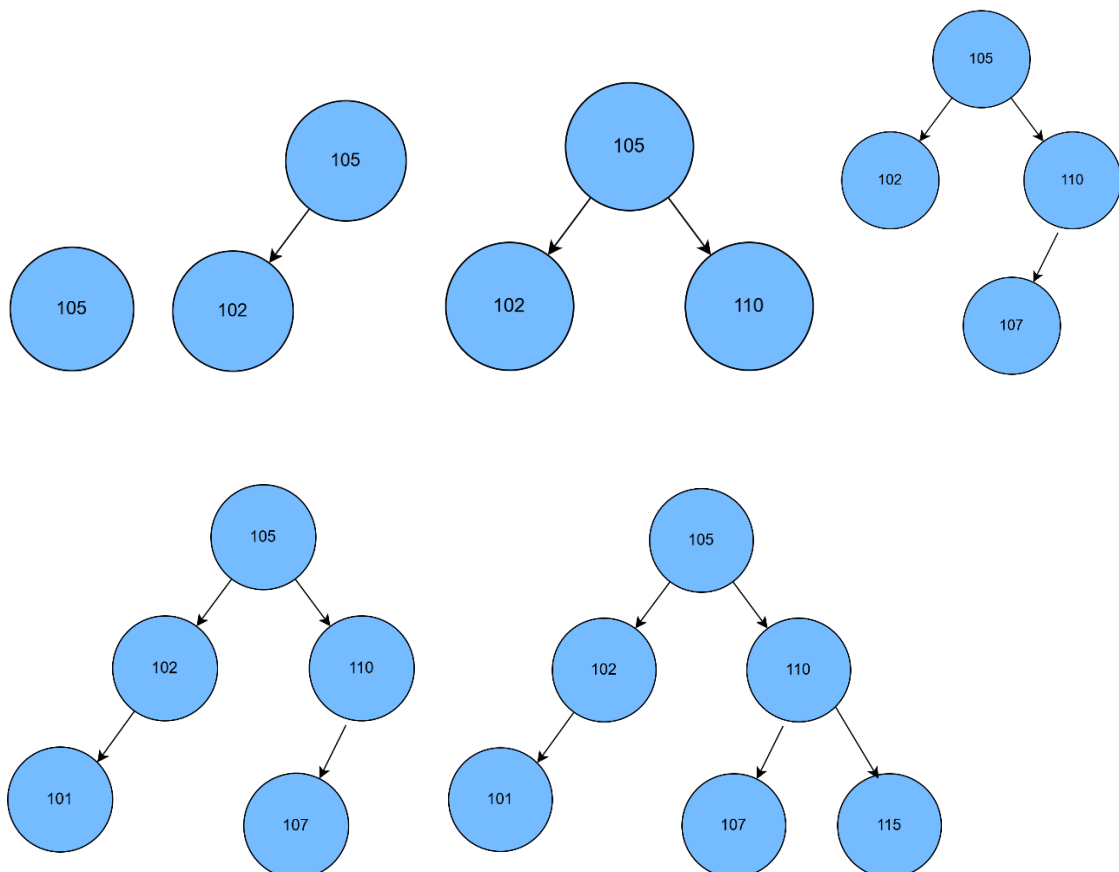
```
def insert_sample_students(student_tree):
    """Inserts 12 sample student records into the BST"""
    students = [
        (105, "Alice Johnson", 3.5),
        (102, "Bob Smith", 3.8),
        (110, "Charlie Brown", 2.9),
        (107, "Diana Prince", 3.7),
```

```

(101, "Ethan Hunt", 3.2),
(115, "Fiona Clark", 3.9),
(108, "George Miller", 3.1),
(104, "Hannah Lee", 3.6),
(112, "Ian Wright", 3.4),
(109, "Jessica Davis", 3.0),
(103, "Kyle Johnson", 3.3),
(106, "Liam Scott", 3.75),
# Add more students as needed
]

```

CodeBlock.4. Represents the number of students present in the student management tree.



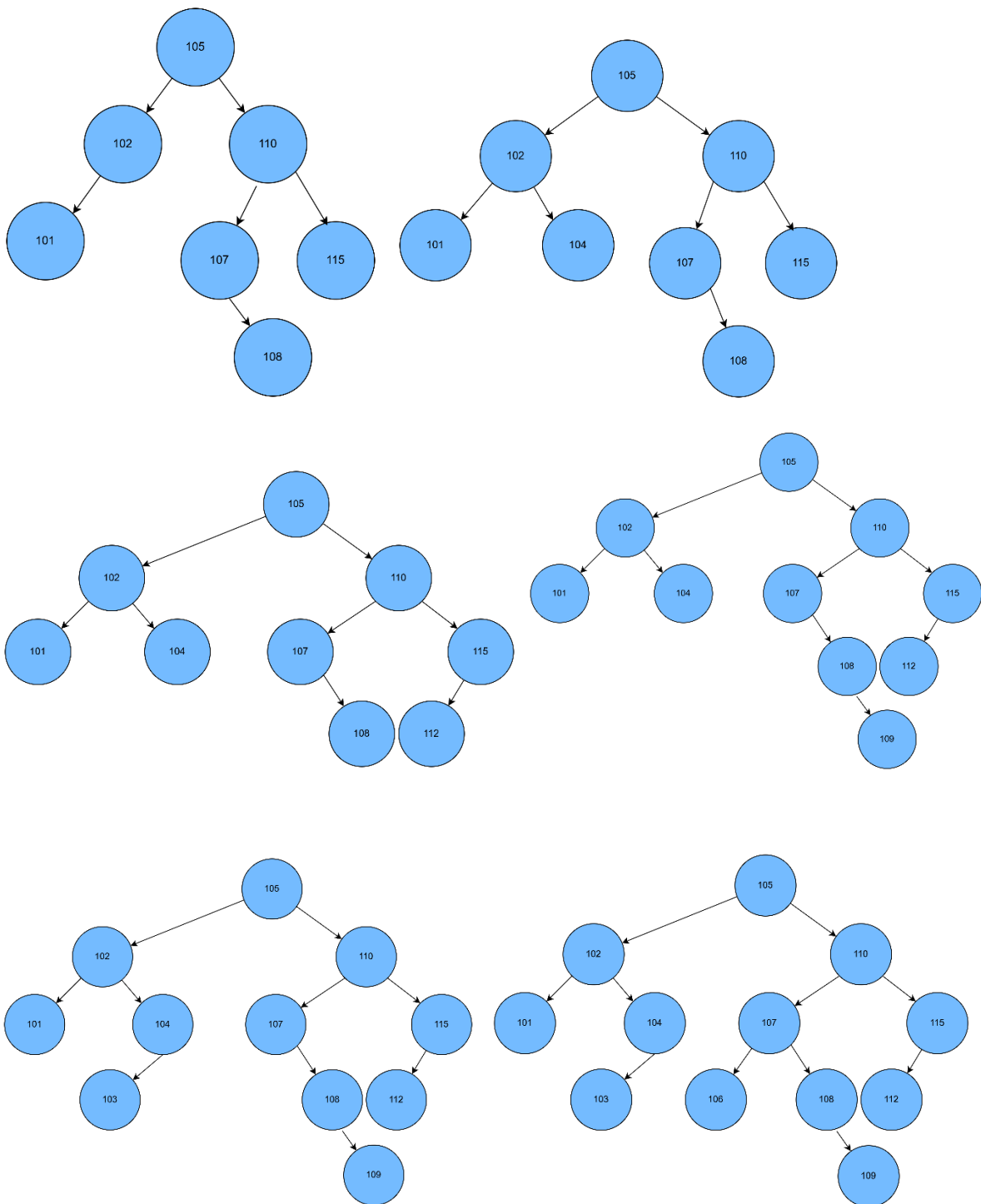


diagram.1-12. Represents the sequential tree diagram showcasing the organisation of 12 student objects them.

SECTION B: ALGORITHM ANALYSIS [25 marks]

B1. Measuring Execution Times and Comparison – Sorting Methods

Refer to Sorting.py

Based on the provided sorting algorithms - **Selection Sort, Quick Sort and Quick Sort modified** complete various tasks in this section (B1). The coding related to computing execution time of algorithms is basic one – for limited size of input. You need to change it according to the requirements of the task.

B1.1 Record average execution times (in microseconds) for Selection Sort and Quick Sort using a randomly generated list and a pre- sorted list. [4 marks]

Detailed Steps:

Run each sorting algorithm **Selection Sort** and **Quick Sort** on two types of lists:

- **List-1: Randomly generated list**
- **List-2: Pre-sorted list**

List sizes to be tested: 5, 10, 100, 1000, 10,000 elements.

Fill in the table (Table-1) with recorded **average execution times**.

Table-1: Average execution times of Sorting methods

Size	Selection Sort (List-1)	Selection Sort (List-2)	QuickSort (List-1)	QuickSort (List2)
5	9μs	4μs	11μs	4μs
10	5μs	3μs	7μs	3μs
100	132μs	103μs	66μs	35μs
1000	10596μs	10188μs	652μs	386μs
10000	1066641μs	1066703μs	6827μs	4403μs

B1.2 Record average execution times (in microseconds) for Quick Sort (modified). [2 marks]

Detailed Steps:

Run Quick Sort (modified-1) the two lists (List-1 and List-2) used in previous step i.e. B1.1.

List sizes to be tested: 5, 10, 100, 1000, 10,000 elements.

Fill in the table (Table-2) with recorded **average execution times**. You need to copy the execution times from previous step i.e. B1.1 in the first four columns.

Table-2: Average execution times of Sorting methods

Size	Selection Sort (List-1)	Selection Sort (List-2)	QuickSort (List-1)	QuickSort (List2)	QuickSort_modified (List-1)	QuickSort_modified (List2)
5	9μs	4μs	11μs	4μs	6μs	2μs
10	5μs	3μs	7μs	3μs	8μs	6μs
100	132μs	103μs	66μs	35μs	51μs	29μs
1000	10596μs	10188μs	652μs	386μs	644μs	19036μs
10000	1066641μs	1066703μs	6827μs	4403μs	6534μs	Error - Recursion Depth Exceeded

B1.3 Based on the execution times recorded in Table-2, Compare the performance of Selection Sort and Quick Sort for different input sizes. How do the actual running times support the theoretical complexities of these sorting algorithms? [12 marks]

Comparison of Selection Sort and QuickSort Performance Based on Table-2

Performance Analysis Across Input Sizes

- **Size 5:**
 - Selection Sort (List-1: 9 μ s, List-2: 4 μ s) is slightly slower than QuickSort (List-1: 11 μ s, List-2: 4 μ s) and QuickSort mod (List-1: 6 μ s, List-2: 2 μ s). All methods are fast due to the small size.
- **Size 10:**
 - Selection Sort (List-1: 5 μ s, List-2: 3 μ s) remains competitive with QuickSort (List-1: 7 μ s, List-2: 3 μ s) and QuickSort mod (List-1: 8 μ s, List-2: 6 μ s), with minimal differences.
- **Size 100:**
 - Selection Sort (List-1: 132 μ s, List-2: 103 μ s) starts to lag behind QuickSort (List-1: 66 μ s, List-2: 35 μ s) and QuickSort mod (List-1: 51 μ s, List-2: 29 μ s), showing QuickSort's efficiency advantage.
- **Size 1000:**
 - Selection Sort (List-1: 10,596 μ s, List-2: 10,188 μ s) is significantly slower than QuickSort (List-1: 652 μ s, List-2: 386 μ s) and QuickSort mod (List-1: 644 μ s, List-2: 19,036 μ s), highlighting QuickSort's scalability.
- **Size 10,000:**
 - Selection Sort (List-1: 1,066,641 μ s, List-2: 1,066,703 μ s) is much slower than QuickSort (List-1: 6,827 μ s, List-2: 4,403 μ s), while QuickSort mod (List-1: 6,534 μ s) performs well, but QuickSort mod (List-2) fails due to recursion depth.

Observation after testing

- Selection Sort shows a steady increase in time with input size, with List-1 and List-2 performing similarly, indicating consistency but poor scalability.
- QuickSort outperforms Selection Sort at larger sizes (100+), with List-2 generally faster than List-1, suggesting better pivot choices or data distribution.
- QuickSort mod varies, with List-1 performing well but List-2 failing at 10,000 due to recursion issues, indicating instability in some cases.

Table: Theoretical Complexities and Actual Running Times

Sorting Algorithm	Theoretical Complexity	Actual Running Times (μ s)	Support from Data
Selection Sort	$O(n^2)$ in all cases, due to comparing each element with every other to find the minimum.	Size 5: 9 (List-1), 4 (List-2) Size 10,000: 1,066,641 (List-1), 1,066,703 (List-2)	Times grow quadratically (e.g., $10,596/132 \approx 80$ for 1000/100 vs. $100^2/10^2 = 100$),

			aligning with $O(n^2)$. List-2 is slightly optimized (e.g., 10,188 vs. 10,596 at 1000).
QuickSort	$O(n \log n)$ average case, $O(n^2)$ worst case, depending on pivot selection.	Size 5: 11 (List-1), 4 (List-2) Size 10,000: 6,827 (List-1), 4,403 (List-2)	Times grow slower (e.g., $652/66 \approx 9.9$ for $1000/100$ vs. $100 \log 1000/100 \log 100 \approx 6.9$), supporting $O(n \log n)$. List-2's better performance (e.g., 386 vs. 652 at 1000) suggests good pivot choices avoiding $O(n^2)$.
QuickSort mod	Similar to QuickSort, but modifications (e.g., pivot strategy) aim to improve average case or avoid recursion depth issues.	Size 5: 6 (List-1), 2 (List-2) Size 10,000: 6,534 (List-1), Error (List-2)	List-1 matches $O(n \log n)$ (e.g., 644 at 1000), but List-2's failure at 10,000 indicates $O(n^2)$ worst case or recursion limit issues, not fully mitigated by modifications.

Conclusion

The actual running times support the theoretical complexities: Selection Sort's $O(n^2)$ is evident in its steep time increase, while QuickSort's $O(n \log n)$ is reflected in its efficiency at larger sizes. QuickSort mod's performance varies, with List-1 supporting $O(n \log n)$ but List-2's error highlighting potential worst-case pitfalls, likely due to poor pivot or recursion handling.

B2. Graphs – Representation and Performance Analysis

StudentFriendshipGraph.py

Given the **student friendship system graph** structure and operations performed in this implementation, **analyse** how the choice of **graph representation** would impact the efficiency of operations like adding friendships and checking a student's friends. **How would changing the representation impact the performance of adding friendships and checking if two students are friends? Discuss the trade-offs and suggest when each representation would be more suitable.**

[4 marks]

In the StudentGraph class, the representation of the graph uses an adjacency list, in that the students are keys in the dictionary and the friends are stored in lists. This representation impacts the operation efficiency for adding friendships and checking friendships in the following ways:

Graph Representation (Adjacency List):

- The **adjacency list** provides an efficient way to represent the graph for sparse graphs (like a friendship network). Each student has a list of friends, and adding a new friendship or checking if two students are friends takes **$O(1)$** and **$O(k)$** time respectively, where **k** is the number of friends the student has.

1. Adding a Friendship

• Adjacency List:

To add a friendship, the algorithm simply adds the students to each other's friend lists. The operation is efficient due to its average time complexity of $O(1)$, as appending to a list takes constant time.

• Adjacency Matrix:

It would be setting two matrix entries and hence $O(1)$ too. However, resizing may cost if it is a new student added.

• Edge List:

This way of adding a friendship will be done by appending the pair of students to the list and costs $O(1)$.

2. Friendship Checking

• Adjacency list:

To check whether two students are friends, the algorithm wanders through the friends' list, which is incurred with a time complexity of $O(k)$ where k denotes the number of friends.

• Adjacency matrix:

Friendships can be checked quickly, with constant time complexity $O(1)$, as we only need a lookup in the matrix.

• Edge list:

To check if two students are friends, the search has to be performed through the edge list, which entails a time complexity of $O(m)$ where m is the number of edges.

Trade-offs and Recommendations:

• Adjacency list:

For sparse or moderately dense graphs. It is memory-efficient, faster in adding friends, and slower for checking if two students are friends.

• Adjacency matrix:

For dense graphs, however, it uses much more memory for sparse graphs.

- **Edge-list:**

Okay for very sparse graphs but inefficient for frequent friendship checking.

Conclusion:

For a sparse graph scenario, adjacency list representations are the most efficient, where adding a friendship will occur more often than checking for a friendship. If the graph were denser, friendship checking might be better done using an adjacency matrix, but this advantage comes at a high cost in terms of memory use.

B3. Analysis - Searching Algorithms [3 marks]

The given **student management system** provides two different methods for searching a student by ID

- **search(self, student_id)**
- **locate_student(self, student_id)**

Analyse and compare the approaches of these two methods.

Comparison of Searching Methods in StudentBST Class

Aspect	search(self, student_id) (Iterative Approach)	locate_student(self, student_id) (Recursive Approach)
Approach	Uses an iterative while loop with a current pointer to traverse the tree.	Uses a recursive helper method (_search_student) to traverse the tree.
Key Steps	1. Initialize current = self.root. 2. Loop: Compare student_id with current.student_id, move left or right, return if matched. 3. Return None if current is None.	1. Call _search_student(self.root, student_id). 2. Base cases: Return None if node is None, return node if IDs match. 3. Recurse on left subtree, then right if not found.
Time Complexity	O(h), where h is the height of the BST (O(log n) for balanced, O(n) for skewed).	O(h), where h is the height of the BST (O(log n) for balanced, O(n) for skewed).
Space Complexity	O(1), uses only a single current pointer, no additional stack space.	O(h), due to the recursive call stack, with maximum depth equal to the tree height.

Performance Considerations	- Faster for deep trees due to no recursion overhead. – - Avoids stack overflow risks. - - Slightly less intuitive code.	- Concise and aligns with tree structure. - Slight overhead from function calls. - Risk of stack overflow in deep trees.
Robustness	Safer for large or unbalanced trees, handles edge cases (empty tree, not found) effectively.	Equally robust for correctness, but vulnerable to stack overflow in skewed trees (e.g., 1,000,000 nodes).
Relevance	Ideal for systems with limited stack space or potentially deep trees.	Suitable for small to medium trees where code clarity is prioritized.
Insight	Efficient and robust, especially when tree balance cannot be guaranteed.	Elegant but requires careful management of tree depth to avoid performance issues.

Conclusion

The search method, with its iterative approach, offers $O(h)$ time and $O(1)$ space complexity, making it more space-efficient and robust for deep or unbalanced trees, avoiding recursion-related risks like stack overflow. The `locate_student` method, with its recursive approach, also achieves $O(h)$ time but uses $O(h)$ space due to the call stack, providing a more readable solution at the cost of potential stack overflow in deep trees. For small to medium-sized balanced trees, both are effective, but search is preferable for larger or skewed trees due to its minimal memory usage and safety. If readability is key and tree depth is controlled, `locate_student` is a viable choice.